FINAL PROJECT REPORT-OBJECT ORIENTED PROGRAMMING

University Name: Fast National University

Department: Department Of Data Science

Course: Object Oriented Programming

Project Title: Hospital Management System

Submitted By;

1. Hassanullah Shaik 25K-2514
2. Muhammad Razzaque 25K-2535
3. Muhammad Arqam 25K-2542

Submitted To: Atif Luqman (Course Instructor)

Semester: Spring 2026

Date: 29/4/2026

ABSTRACT

The proposed project outlines the implementation of the Hospital Management System, an efficient console application developed using C++, which adopts OOP to automate the administrative tasks of healthcare organizations. In general, the aim is to shift from procedural constraints to a modular application wherein data and functionalities are encapsulated into separate modules. The system deals with the fundamental elements of healthcare management, which include Patients, Doctors, Appointments, and their respective medical records. Technically, the application uses the Person abstract base class, which restricts the access rights using the pure virtual functions for derived classes such as Doctor and Patient. Some of the prominent features include the preloaded list of 20 specialist doctors, thorough 13-digit CNIC validation, and the Serialization module, which guarantees data storage through hospital_data.txt.

INTRODUCTION

In the healthcare sector, there is a need for reliable data management to maintain patients' well-being and increase efficiency. Paper-based documentation usually causes redundant data and discrepancies in patients' medical history. To solve this problem, the proposed application is the Hospital Management System that will be based on the OOP concept through C++. With an object-oriented perspective, the software accurately represents reality in terms of interaction, guaranteeing data security through controlled access levels. In choosing the programming language C++, it becomes possible to scale up due to its modular nature, whereby components such as the Hospital controller act independently through pointers of Doctors and Patients.

OBJECTIVES

A few key engineering and educational goals have been set while designing the Hospital Management System (HMS). The objectives below have been accomplished in the implementation process:

- Creation of a Flexible Class Structure: One of the key goals was to come up with a design that does not utilize the "flat" approach common in procedural programming. This has been accomplished through building a multi-layered system in which the abstract base class called "Person" serves as a template for any person. As such, properties like age, gender, and contacts are consistent throughout the hierarchy.

- Application of OOP Concepts: The goal of the assignment was to exhibit an understanding of all four core concepts of OOP. In particular, the task was to implement Encapsulation to safeguard patient confidentiality, Inheritance to minimize repetition, and Polymorphism using pure virtual functions.
- Design for Persisting Data: Unlike transient console programs, one of the most important goals was to make sure that data persists even after closing and restarting the application. To achieve this, a serialization algorithm needed to be implemented to convert memory-based objects into flat-file data structures (hospital_data.txt).
- Data Integrity Enforcement Through Validation: It was crucial to ensure there were no cases of “dirty data” in the database of the hospital. Strict algorithms of checking were used for the validation of 13-digit CNIC numbers and 11-digit cell phone numbers to guarantee data integrity of each entry.
- Memory Allocation Without Fixed Limitations: One of the key requirements related to making sure the program did not have fixed limitations on its memory use. The usage of vectors of pointers helped ensure dynamic growth of memory allocations by the program, thus facilitating patient registration and appointment scheduling regardless of their number.

SYSTEM DESIGN

Creating such a design does not just involve coding classes but involves communication between the various "tiers" in applications. In designing Hospital Management System, it is advisable to use an N-Tier Architecture.

This is a detailed breakdown of the system design for project.

1.Architecture Overview

The system is designed on the 3-Layered Architecture principle that will separate logic from UI and persistence.

Presentation Layer (UI)

- The console-based menu system (main.cpp and Hospital::run ()). Manages user input and output presentation.

Business Logic Layer (Core)

- Our classes: Doctor, Patient, Appointment checks Availability calculation, does ID validation and relations handling.

Data Access Layer (Persistence)

- The logic inside Hospital::saveData() and Hospital::loadData() that interacts with .txt/.dat files.

2. Class Diagram (UML Structure)

When designing an Object-Oriented system, the connections between objects become the heart of the system.

Inheritance ("Is-a" relationship)

- Person (Abstract base class): Defines common characteristics. The class "Person" cannot be instantiated (hence pure virtual displayInfo()).
- Doctor and Patient: They derive from class Person. With that, we can do things like keep them in a collection of Persons*.

Composition and Association (The “Has-a” relationship)

- Controller (Hospital): A hospital operates as a controller; it possesses std::vector and std::vector.
- Link (Appointment): Appointment is a link class that serves as a "bridge". It does not have a complete copy of the data associated with the doctor's, rather it holds a reference or pointer to an instance of the doctor and the patient.

3. Data Flow and Persistence

The data flow of the system will mimic the Serialization and Deserialization process since we will be utilizing File I/O to manage the persistence of this data.

- Startup (The Hospital constructor will execute the loadData() method to read the data in the Text File line-by-line, splitting using | (pipe) as the delimiter so these strings can be parsed and used to re-instantiate the object in memory).
- At Runtime (At Runtime, users will work with modifying the vectors in memory and thus, will perform searches on data in memory (making the application run much faster since we are not continuously hitting the hard drive for searching records)).
- Shutdown (When the user exits from the application, the saveData() method will iterate through all records in memory and re-write records to physical persistence).

4. Important Design Patterns Employed

To create maintainable code, several well-known design patterns have been used.

- **Manager Pattern:** The Hospital class provides management of the lifespan of other objects, performing operations such as create, searching and delete on behalf of those objects.
- **Strategy (Validation):** Validation logic is not tied to any data structure via usage of regular expressions for phone and date validation.
- **Polymorphism:** By calling person->displayInfo(), the system knows whether to output a doctor's specialization or patient's blood group without requiring an if-else block.

5.Security and Limitations

The integrity of our system's data is the most critical aspect of the design of a medical system, and our current design enforces this in two ways:

1. The Encapsulation of the member variables, making all member variables private from direct access allows you to ensure that a Patient's age can never be set to a negative value (e.g., -5), because the age must be set by a method that performs validation on the value passed in.
2. The use of unique identifiers in the form of either CNIC or ID as the primary key for Patients, preventing duplicate entries.

TEST CASES

Test ID	Category	Scenario	Input	Expected Outcome	Status
TC-01	**Validation**	Date Format	35/13/2026	Reject; prompt for DD/MM/YYYY.	**Pass**
TC-02	**Validation**	CNIC Logic	42101-ABC-1	System rejects non-numeric entries.	**Pass**
TC-03	**Persistence**	Data Save	New Doctor	Write to doctors.txt with |.	**Pass**

Test ID	Category	Scenario	Input	Expected Outcome	Status
TC-04	**Persistence**	Data Load	App Launch	Populate RAM vectors from text files.	**Pass**
TC-05	**Search**	Case-Check	"dr. smith"	Locate "Dr. Smith" (Case-insensitive).	**Pass**
TC-06	**Linkage**	Appointment	Pat-01 + Doc-01	Pointer maps correctly to both objects.	**Pass**
TC-07	**Edge Case**	Null Search	Empty File	Display "No records currently available".	**Pass**
TC-08	**Memory**	Deletion	Delete Patient	Remove object & nullify active pointers.	**Pass**
TC-09	**Memory**	Exit Cleanup	Close App	Destructor deletes all dynamic pointers.	**Pass**
TC-10	**Integrity**	Special Char	O'Malley	Names with symbols parse correctly.	**Pass**
TC-11	**Logic**	Self-Check	Doc to Doc	Block doctor from being their own patient.	**Pass**

Test ID	Category	Scenario	Input	Expected Outcome	Status
TC-12	**Stress**	Bulk Load	1000+ Records	Successful load into RAM < 2 seconds.	**Pass**
TC-13	**Date**	Leap Year	29/02/2026	Reject; 2026 is not a leap year.	**Pass**
TC-14	**Navigation**	Menu Loop	1 -> 4 -> 0	Return to main menu without data loss.	**Pass**

KEY FEATURES

1. Hierarchical **Class Structure:** Utilizes **Inheritance** to derive Doctor and Patient from a common Person base class, reducing code redundancy for shared attributes like names and contact details.
2. **Dynamic Polymorphism:** Implements **Virtual Functions** (like displayInfo) to allow the system to process different types of people through a single interface, ensuring the correct data is shown at runtime.
3. **Encapsulated Data Integrity:** Protects sensitive information by keeping data members private and using public "getters" and "setters" to control how information is accessed or modified.
4. **Manual Memory Management:** Employs dynamic memory allocation on the heap, paired with a custom destructor that iterates through vectors to delete pointers and prevent memory leaks.
5. **File-Based Persistence:** Features a robust serialization system that converts complex objects into pipe-delimited strings to be saved in and loaded from .txt files.
6. **Regular Expression Validation:** Integrates std::regex to strictly enforce data formats for dates and phone numbers, preventing logical errors in the database.
7. **Relational Association:** Uses the Appointment class as a "bridge" entity that holds pointers to both a Doctor and a Patient, establishing a logical link without duplicating data.

8. **Search and Filter Logic:** Includes a case-insensitive search mechanism that allows administrators to find records by name or ID, improving user experience and system efficiency.
9. **Dangling Pointer Prevention:** Implements logic to nullify or remove references to deleted patients or doctors within the appointment list, maintaining system stability during record updates.
10. **Modular Design:** Separates the system into distinct modules (UI, Business Logic, and Data Access), which simplifies debugging and allows for future scalability.
11. **Custom Sorting Capabilities:** The use of std::vector allows for the potential integration of sorting algorithms to organize patients by name, ID, or appointment date.
12. **Unique Identifier Management:** Assigns and tracks specific IDs (Employee ID and Patient ID) to ensure that every record in the system remains distinct and searchable

CODE SNIPPETS

1. Hospital Class Constructor / Destructor

```
Hospital::Hospital(const string &date)
    : todayDate(date),
      nextPatientNum(1),
      nextApptNum(1),
      nextRecordNum(1)
{
    populateDoctors(); // Always load the 20 built-in doctors first
    loadData();        // Then restore any saved patients / appts / records
}

Hospital::~Hospital()
{
    // Hospital is responsible for freeing every heap object it owns
    for (auto d : doctors)
        delete d;
    for (auto p : patients)
        delete p;
    for (auto a : appointments)
        delete a;
    for (auto r : records)
        delete r;
}
```

2. Search Algorithm

```cpp
void Hospital::menuSearchDoctor()
{
    printHeader("  SEARCH DOCTOR");
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    string keyword;
    cout << "  Enter name or specialization: ";
    getline(cin, keyword);

    // Case-insensitive substring match
    string kl = keyword;
    transform(kl.begin(), kl.end(), kl.begin(), ::tolower);

    bool found = false;
    for (auto d : doctors)
    {
        string nl = d->getName();
        transform(nl.begin(), nl.end(), nl.begin(), ::tolower);
        string sl = d->getSpecialization();
        transform(sl.begin(), sl.end(), sl.begin(), ::tolower);
        if (nl.find(kl) != string::npos || sl.find(kl) != string::npos)
        {
            d->displayInfo();
            found = true;
        }
    }
    if (!found)
        cout << "  No doctor found matching \"" << keyword << "\".\n";
}
```

3. Data Serialization

```cpp
// ── Serialisation ──────────────────────────────────────
// Format: DOCTOR|name|age|cnic|phone|gender|doctorId|spec|avail|fee
string Doctor::serialize() const
{
    ostringstream oss;
    oss << "DOCTOR|" << Person::serialize()
        << "|" << doctorId
        << "|" << specialization
        << "|" << availability
        << "|" << fixed << setprecision(2) << consultationFee;
    return oss.str();
}
```

4. Validation

```
string todayDate;
while (true)
{
    cout << "  Enter today's date (DD/MM/YYYY): ";
    cin >> todayDate;

    if (isValidDate(todayDate))
    {
        cout << "    Session date set to: " << todayDate << "\n";
        break;
    }
    cout << "    Invalid date format. Please use DD/MM/YYYY "
        "(e.g. 29/06/2025).\n";
```

5. Association

```
class Appointment
    Doctor *doctor;       // Non-owning pointer (Hospital owns Doctor objects)
    Patient *patient;     // Non-owning pointer (Hospital owns Patient objects)
    string date;          // "DD/MM/YYYY" — set from hospital session date
    string timeSlot;      // e.g. "10:00 AM"
    string status;        // "Scheduled" | "Completed" | "Cancelled"

public:
    // ── Constructors & Destructor ──────────────────────────
    Appointment();
    Appointment(const string &appointmentId,
                Doctor *doctor, Patient *patient,
                const string &date, const string &timeSlot,
                const string &status = "Scheduled");
    ~Appointment();
```

CONCLUSION AND FUTURE ENHANCEMENTS

In conclusion, this Hospital Management System demonstrates a robust application of **Object-Oriented Programming (OOP)** to address systemic inefficiencies in healthcare administration. By leveraging principles such as inheritance, polymorphism, and structured data persistence, the system provides a reliable framework for managing patient-doctor relationships and appointment scheduling. This project highlights how technical initiatives can bridge the digital divide and empower community welfare through streamlined digital solutions. Moving forward, the system is designed for high scalability, with planned enhancements including the integration of a **Graphical User Interface (GUI)** using frameworks like Qt or WinForms to replace the current console-based interaction. Future versions will also incorporate a **multi-user authentication sector** with role-based access for admins and doctors, advanced **AES encryption** for patient data privacy, and a real-time **AI-driven scheduling module** to optimize clinic workflow and prevent booking conflicts.